# Práctica 6: Clasificación de Texto y Filtros Anti-Spam (Naïve Bayes)

**Objetivo:** Comprender cómo un modelo probabilístico puede clasificar texto a partir de la frecuencia de las palabras. En particular, se estudiará cómo representar mensajes mediante una bolsa de palabras, cómo aplica el clasificador Naïve Bayes el Teorema de Bayes para distinguir entre mensajes legítimos y spam, y por qué el suavizado de Laplace resulta esencial cuando aparecen términos no vistos durante el entrenamiento.

## Material de partida

Se proporciona lo siguiente:

- Un dataset real de mensajes SMS etiquetados como `ham` o `spam` (descargado de Kaggle)
- Una plantilla de código base en Python con `pandas` y `scikit-learn`, para facilitar la carga de datos, la vectorización del texto y el entrenamiento inicial del modelo.
- Un problema de clasificación binaria en el que el objetivo es decidir si un mensaje es legítimo o no deseado a partir de su contenido textual.

### Nota

En esta práctica se proporcionará una implementación base para acelerar el trabajo mecánico. El objetivo no es que construyas un clasificador probabilístico desde cero, sino que entiendas qué información utiliza el modelo, cómo transforma el texto en números y cómo interpreta las palabras para tomar decisiones.

## Introducción

En problemas de clasificación de texto, los datos de entrada no son números ni vectores geométricos sencillos, sino mensajes escritos en lenguaje natural. Antes de poder aplicar un algoritmo de aprendizaje automático, es necesario transformar ese texto en una representación numérica que el modelo pueda manejar.
Una estrategia clásica consiste en usar una **bolsa de palabras** (*Bag of Words*), donde cada mensaje se representa mediante el recuento de las palabras que contiene. A partir de esa representación, un clasificador como **Naïve Bayes** puede estimar qué palabras aparecen con mayor frecuencia en los mensajes spam y cuáles son más habituales en mensajes legítimos.
Este enfoque es especialmente interesante porque, aunque es sencillo, permite introducir varias ideas fundamentales del aprendizaje automático:

- la necesidad de preprocesar los datos antes de entrenar un modelo;
- la conversión de información textual en variables numéricas;
- el uso de probabilidades para clasificar ejemplos;
- y la importancia de evitar problemas numéricos mediante técnicas como el **suavizado de Laplace**.

En esta práctica vas a trabajar con estas ideas de forma experimental, observando cómo un modelo puede aprender a detectar patrones de lenguaje asociados al spam.

## Tarea 1: Del texto a los números

En esta primera parte vas a transformar los mensajes en una representación que el modelo pueda utilizar.

### Qué debes hacer

- Carga el dataset de mensajes SMS proporcionado.
- Convierte las etiquetas del problema a formato numérico:
  - `ham` $\to 0$;
  - `spam` $\to 1$.
- Aplica un preprocesamiento básico al texto:
  - pasar todo a minúsculas;
  - eliminar signos de puntuación u otros símbolos irrelevantes.

- Divide los datos en entrenamiento y prueba.
- Utiliza una representación de tipo **Bag of Words** para convertir cada mensaje en un vector de frecuencias de palabras.

#### Cuestión

Debes explicar por qué esta transformación es necesaria.
No basta con indicar que el algoritmo "necesita números". Debes razonar qué información conserva una representación basada en frecuencias de palabras y qué tipo de información se pierde al ignorar el orden exacto de las palabras en la frase.

#### Reflexión

Comenta qué significa que en la matriz resultante:

- cada fila represente un mensaje;
- cada columna represente una palabra del vocabulario;
- y cada valor indique cuántas veces aparece esa palabra en ese mensaje.

El objetivo de esta parte es que entiendas que la representación de los datos condiciona completamente lo que el modelo puede aprender.

### Tarea 2: Entrenando el clasificador Naïve Bayes

Una vez convertidos los mensajes en vectores de frecuencias, vas a entrenar un modelo probabilístico para distinguir entre ham y spam.

#### Qué debes hacer

- Entrena un clasificador **Multinomial Naïve Bayes**.
- Configura el parámetro `alpha = 1.0`.
- Ajusta el modelo con los datos de entrenamiento.
- Utiliza el modelo para predecir las etiquetas del conjunto de prueba.

#### Qué debes interpretar

Debes explicar qué significan las dos ideas probabilísticas centrales del modelo:

- la **probabilidad a priori**, es decir, la proporción global de mensajes spam y ham en el conjunto de entrenamiento;
- la **probabilidad condicional**, es decir, la probabilidad de observar una palabra concreta sabiendo que el mensaje pertenece a una clase determinada.

Por ejemplo, debes ser capaz de interpretar de forma razonada una idea como:

$$P(\text{“gratis”} \mid \text{spam})$$

y explicar qué información aporta al modelo.

#### Análisis

Debes discutir también por qué este modelo recibe el nombre de *Naïve*.
En particular, debes comentar que el clasificador asume una independencia simplificada entre palabras, aunque en lenguaje real esa independencia no se cumpla estrictamente. El interés aquí no es demostrar formalmente esta hipótesis, sino entender que se trata de una simplificación útil que permite construir un clasificador eficaz y fácil de entrenar.

## Tarea 3: El papel del suavizado de Laplace

En esta parte estudiarás una cuestión clave: qué ocurre cuando el modelo encuentra una palabra que no había visto antes en el entrenamiento.

### Qué debes hacer

- Explica qué problema aparece cuando una palabra tiene frecuencia cero en una determinada clase.
- Justifica por qué el modelo incorpora el parámetro `alpha = 1.0`.
- Relaciona esta corrección con el llamado **suavizado de Laplace**.

### Cuestión

Debes responder razonadamente a esta pregunta:
**¿Qué ocurriría si intentáramos clasificar una palabra como "oferta" y esa palabra nunca hubiera aparecido en el entrenamiento?**
Tu explicación debe dejar claro por qué, sin suavizado, una probabilidad nula podría arruinar el cálculo completo del modelo.

### Interpretación

No se trata solo de decir que "se suma 1" a todos los recuentos. Debes explicar el sentido práctico de esa operación:

- evitar probabilidades exactamente nulas;
- permitir que el modelo sea más robusto ante vocabulario no observado;
- y mantener una estimación razonable incluso cuando los datos son incompletos.

## Tarea 4: Evaluación del modelo

Ahora vas a comprobar cómo se comporta el clasificador sobre ejemplos no vistos.

### Qué debes hacer

- Calcula la **matriz de confusión**.
- Obtén métricas de evaluación como precisión, exhaustividad y *f1-score*.
- Interpreta los aciertos y errores del clasificador.

### Qué debes observar

Analiza qué significado tienen los distintos tipos de error en este problema:

- un mensaje spam clasificado como ham;
- un mensaje legítimo clasificado como spam.

Debes comentar cuál de estos dos errores te parece más problemático en un filtro anti-spam real y por qué.

### Análisis

No basta con copiar la matriz o las métricas. Debes interpretar qué indican sobre el comportamiento del modelo:

- si detecta bien la mayoría del spam;
- si penaliza demasiado mensajes legítimos;
- o si parece conservador a la hora de marcar mensajes sospechosos.

## Tarea 5: Inspeccionando qué ha aprendido el modelo

En esta parte vas a analizar qué palabras considera el clasificador más representativas del spam.

### Qué debes hacer

- Extrae del modelo las probabilidades asociadas a las palabras del vocabulario.
- Identifica las **5 palabras** que el modelo considera más indicativas de la clase spam.
- Comenta si esas palabras te parecen razonables o esperables.

### Reflexión

Debes discutir hasta qué punto esta inspección permite interpretar el comportamiento del clasificador.
Puedes apoyarte en cuestiones como:

- si el modelo parece aprender patrones comprensibles para una persona;
- si algunas palabras tienen sentido claramente comercial o promocional;
- y si el vocabulario aprendido refleja realmente diferencias entre spam y mensajes normales.

## Reto: Diseñar mensajes de prueba

Imagina ahora que quieres poner a prueba el clasificador con ejemplos inventados por ti.

### Qué debes hacer

Debes crear al menos tres mensajes nuevos:

- un mensaje claramente **ham**;
- un mensaje claramente **spam**;
- y un mensaje **ambiguo**.

A continuación, debes usar el modelo entrenado para estimar:

- la clase predicha;
- y la probabilidad asignada a cada clase.

### Cuestiones

Debes responder razonadamente a estas preguntas:

- ¿Coincide la predicción del modelo con lo que esperabas?
- ¿En qué caso parece estar más seguro?
- ¿Qué características del mensaje ambiguo podrían explicar la duda del modelo?

### Interpretación

El objetivo de esta parte es que entiendas que clasificar texto no consiste solo en generar una etiqueta final, sino también en analizar el nivel de confianza del modelo y el tipo de evidencia lingüística que utiliza.

## Formato de entrega

Debes entregar un **Informe de Resultados (PDF)** que incluya:

- una breve explicación de cómo se ha transformado el texto en una representación numérica;
- una descripción del modelo Naïve Bayes utilizado;
- una explicación clara del suavizado de Laplace;
- la matriz de confusión y las métricas principales de evaluación;
- las 5 palabras que el modelo considera más características del spam;
- el análisis de los tres mensajes de prueba creados por ti;
- y una conclusión final sobre las ventajas y limitaciones de este enfoque.

## Recomendaciones

A la hora de redactar el informe, evita limitarte a pegar capturas de código o salidas automáticas sin explicación. Debes interpretar lo que ocurre en cada fase:

- por qué el texto necesita convertirse en números;
- qué aprende realmente un modelo basado en frecuencias de palabras;
- por qué el suavizado de Laplace es necesario;
- cómo deben interpretarse los errores del clasificador;
- y hasta qué punto las palabras más probables permiten entender la decisión del modelo.

El informe debe ser claro, sintético y razonado. Deben priorizarse los resultados, las evidencias experimentales y su interpretación.

## Objetivo final de la práctica

El objetivo final de esta práctica es que entiendas que incluso un modelo muy sencillo puede resolver un problema real de clasificación de texto si los datos se representan adecuadamente. Al mismo tiempo, debes comprender que su funcionamiento depende de hipótesis concretas, como la independencia aproximada entre palabras, y de decisiones técnicas importantes, como la forma de vectorizar el texto o el uso del suavizado de Laplace.